

spark basics

Samradni Dahiphale · 18 July



Week 5 - Apache Spark (PySpark) | Data Cleaning, Transformation & Aggregation

This notebook loads the Kaggle Employee dataset placed in the repo under `Employee/Employee.csv` (as you mentioned: `Employee/Employee.csv`). It performs: duplicates removal, null handling (fillna), schema casting, filtering, column renaming, aggregation, groupBy, and saves results to `output/`.

```
In []:
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName('Celebal-Week5-Spark-Assignment').getOrCreate()

print('Spark Started')
print('Spark version:', spark.version)
```

```
Out []:
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[2], line 1
----> 1 from pyspark.sql import SparkSession
      2
      3 spark = (
      4     SparkSession.builder

ModuleNotFoundError: No module named 'pyspark'
```

```
In []:
# --- Load CSV ---
# Dataset Location: top-level `Employee/Employee.csv`
input_path = '../Employee/Employee.csv'

df = (
    spark.read
        .option('header', True)
        .option('inferSchema', True)
        .csv(input_path)
)

print('Loaded rows:', df.count())
print('Columns:', df.columns)
df.show(5, truncate=False)
df.printSchema()
```

```
In []:
# --- Data Cleaning: Remove duplicates ---
df_clean = df.dropDuplicates()
print('Rows after dropDuplicates():', df_clean.count())
```

```
In []:
# --- Data Cleaning: Handle nulls (fillna) ---
# Strategy: fill numeric nulls with 0 and string nulls with 'Unknown'.

numeric_cols = [t[0] for t in df_clean.dtypes if t[1] in ('int', 'bigint', 'double', 'float', 'decimal')]
string_cols = [t[0] for t in df_clean.dtypes if t[1] == 'string']

fill_map = {c: 0 for c in numeric_cols}
fill_map.update({c: 'Unknown' for c in string_cols})

df_clean = df_clean.fillna(fill_map)
```

```

null_counts = df_clean.select([F.sum(F.col(c).isNull().cast('int')).alias(c) for c in df_clean.columns])
null_counts.show(truncate=False)

```

```

In [ ]:
# --- Schema fixes (casting) ---
# Common numeric columns in employee datasets (adjust if needed).
likely_numeric = ['Age', 'Salary', 'MonthlyIncome', 'YearsAtCompany', 'YearsExperience']
for c in likely_numeric:
    if c in df_clean.columns:
        df_clean = df_clean.withColumn(c, F.col(c).cast('double'))

print('Post-cast schema:')
df_clean.printSchema()

```

```

In [ ]:
# --- Filtering ---
filtered = df_clean

if 'Age' in df_clean.columns:
    filtered = filtered.filter(F.col('Age') >= 25)

# Choose the first available categorical filter column.
cat_col = None
for c in ['Department', 'Region', 'Category', 'JobRole']:
    if c in df_clean.columns:
        cat_col = c
        break

if cat_col:
    filtered = filtered.filter(F.col(cat_col).isNotNull())

filtered.show(10, truncate=False)
print('Filtered rows:', filtered.count())

```

```

In [ ]:
# --- Transformations: rename columns (example) ---
# Rename Salary -> MonthlySalary if applicable
if 'Salary' in filtered.columns and 'MonthlySalary' not in filtered.columns:
    filtered = filtered.withColumnRenamed('Salary', 'MonthlySalary')

# If Age exists and you want a cleaner name
if 'Age' in filtered.columns and 'EmployeeAge' not in filtered.columns:
    filtered = filtered.withColumnRenamed('Age', 'EmployeeAge')

filtered.show(5, truncate=False)

```

```

In [ ]:
# --- Basic Aggregations ---
print('Total (filtered) employees:', filtered.count())

# Choose a salary-like numeric column
salary_col = None
for c in ['MonthlySalary', 'Salary', 'MonthlyIncome']:
    if c in filtered.columns:
        salary_col = c
        break

if salary_col:
    filtered.select(
        F.count(F.lit(1)).alias('row_count'),
        F.avg(F.col(salary_col)).alias('avg_salary'),
        F.min(F.col(salary_col)).alias('min_salary'),
        F.max(F.col(salary_col)).alias('max_salary'),
        F.sum(F.col(salary_col)).alias('sum_salary')
    ).show(truncate=False)
else:
    print('No Salary-like column found for min/max/avg/sum aggregation.')

```

```

In []:
# --- Group By Aggregations ---

group_col = None
for c in ['Department', 'Region', 'Category', 'JobRole']:
    if c in filtered.columns:
        group_col = c
        break

# Use the renamed column if present
salary_col = None
for c in ['MonthlySalary', 'Salary', 'MonthlyIncome']:
    if c in filtered.columns:
        salary_col = c
        break

if group_col and salary_col:
    result = (
        filtered.groupBy(group_col)
        .agg(
            F.count(F.lit(1)).alias('employee_count'),
            F.avg(F.col(salary_col)).alias('avg_salary')
        )
        .orderBy(F.desc('employee_count'))
    )
    result.show(50, truncate=False)

# HAVING-like filter
result_filtered = result.filter(F.col('avg_salary') > 50000)
result_filtered.show(truncate=False)
else:
    print('Required columns for groupBy not found. group_col:', group_col, 'salary_col:', salary_col)

```

```

In []:
# --- Save Outputs as CSV ---
# Spark writes CSV into folders; we remove the `.csv` suffix by slicing off last 4 chars.

output_clean_path = '../output/cleaned_data.csv'
output_grouped_path = '../output/grouped_results.csv'

(
    df_clean.coalesce(1)
    .write
    .mode('overwrite')
    .option('header', True)
    .csv(output_clean_path[:-4])
)

if 'result' in locals():
    (
        result.coalesce(1)
        .write
        .mode('overwrite')
        .option('header', True)
        .csv(output_grouped_path[:-4])
    )

print('Saved outputs to output/ (Spark writes folders).')

```